Функция потерь и производная

Модель и функция потерь

При обучении любой модели мы подбирали идеальные веса. Если это **модель** линейной регрессии (linear regression model) с одним признаком (одной независимой переменной), то наше уравнение может выглядеть следующим образом (постоянный коэффициент или сдвиг мы пока опустим для простоты).

$$y = w \times x$$

Для того чтобы подобрать вес w, у нас есть ещё одна функция, называемая **функцией потерь** (loss function). Это наш критерий качества первой, основной функции модели.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

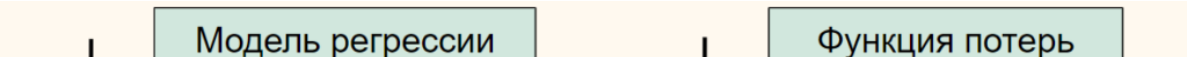
Напомню, в данном случае мы находим среднеквадратическую ошибку или MSE, которая показывает среднее расстояние всех точек до линии регрессии.

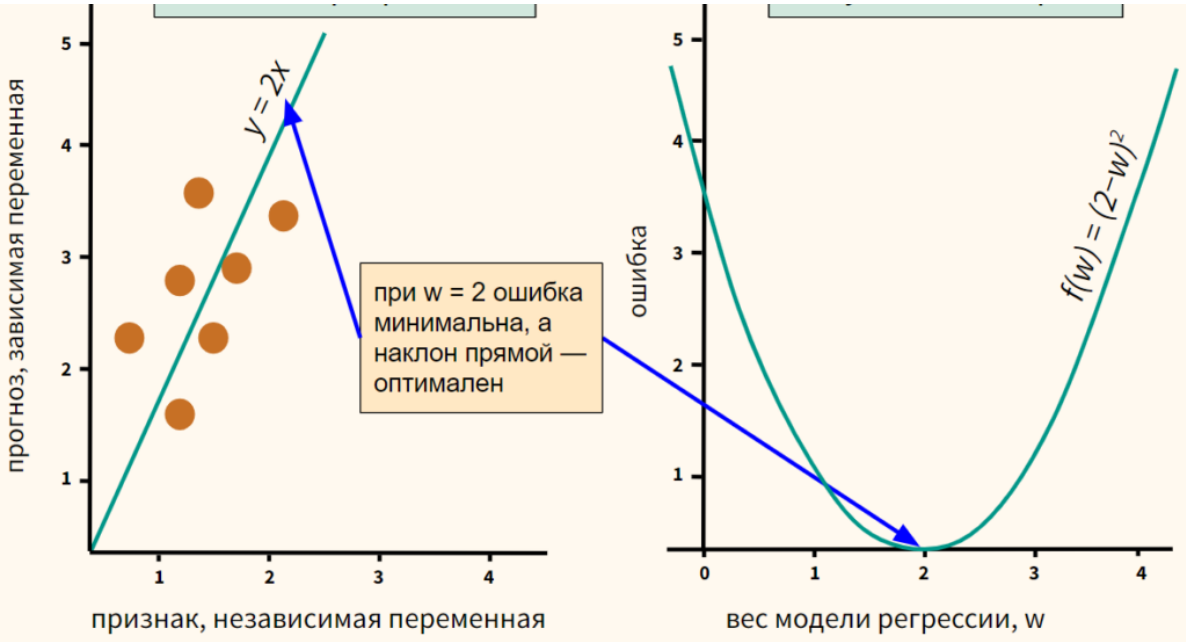
Для того чтобы сделать более очевидной связь функции потерь с исходной моделью, мы можем переписать ее следующим образом.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - (w \times x_i))^2$$

Подставляя наши данные x_i и веса w и сравнивая результат с целевым значением y_i, мы будем на каждой итерации получать определенный уровень ошибки.

Графически, модель регрессии — это прямая, функция потерь — парабола. Представим, что в упрощенном виде, эти функции выглядят следующим образом.



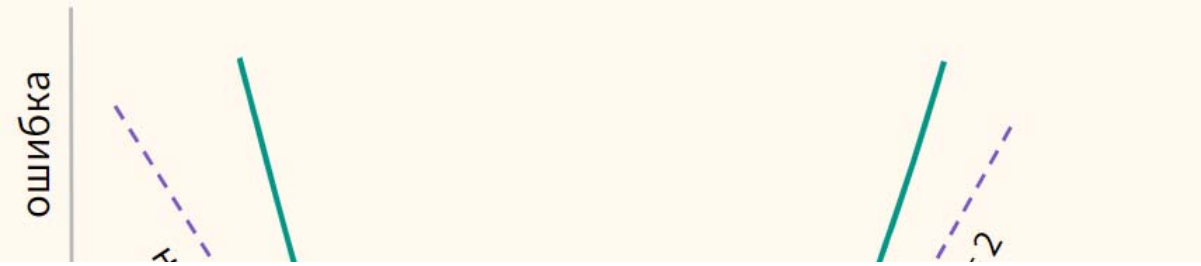


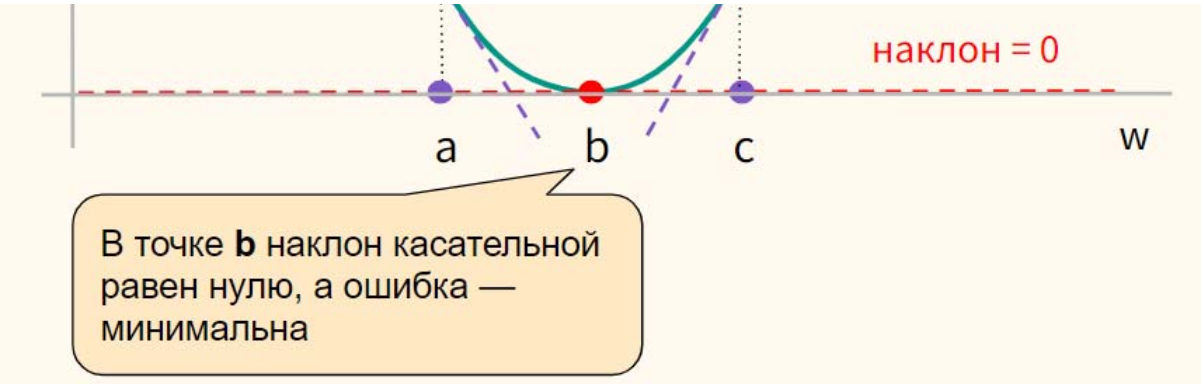
Как они связаны? Там где при определенном весе w значение функции потерь (т.е. ошибка) минимально, вес (а значит и наклон прямой) основной функции оптимален. Из графика очевидно, что идеальным весом с минимальной ошибкой, а значит и наклоном линии регрессии будет $w = 2$.

Как же найти эту точку? Ведь графики — это просто иллюстрация, и никакого надежного метода мы пока не придумали.

Наклон касательной к функции потерь

Давайте возьмем несколько точек на горизонтальной оси w, на уровне этих точек проведем касательные линии (tangent lines) к нашей параболе и рассчитаем их наклон.





Становится очевидно, что в точке b, где наклон касательной равен нулю, наша функция имеет минимальную ошибку, а вес модели идеален. Остаётся выяснить, как рассчитать наклон в каждой из точек.

Производная функции

Оказывается, для многих функций можно найти производную от них функцию или просто **производную** (derivative).

> Значение производной показывает **наклон** касательной к исходной функции в определенной точке.

Давайте разберем это *первое* важное для нас утверждение пошагово:

Шаг 1. Берем функцию, например, параболу и находим еще одну функцию, производную от нее.

$$f(w) = (2 - w)^2$$

$$f'(w) = 2w - 4$$

Обозначение производной $f'(w)$ читается как «эф штрих от далб ю».

Шаг 2. Берем любое значение w, например, $w = 3$, и подставляем в уравнение производной.

$$f'(3) = 2 \times 3 - 4 = 2$$

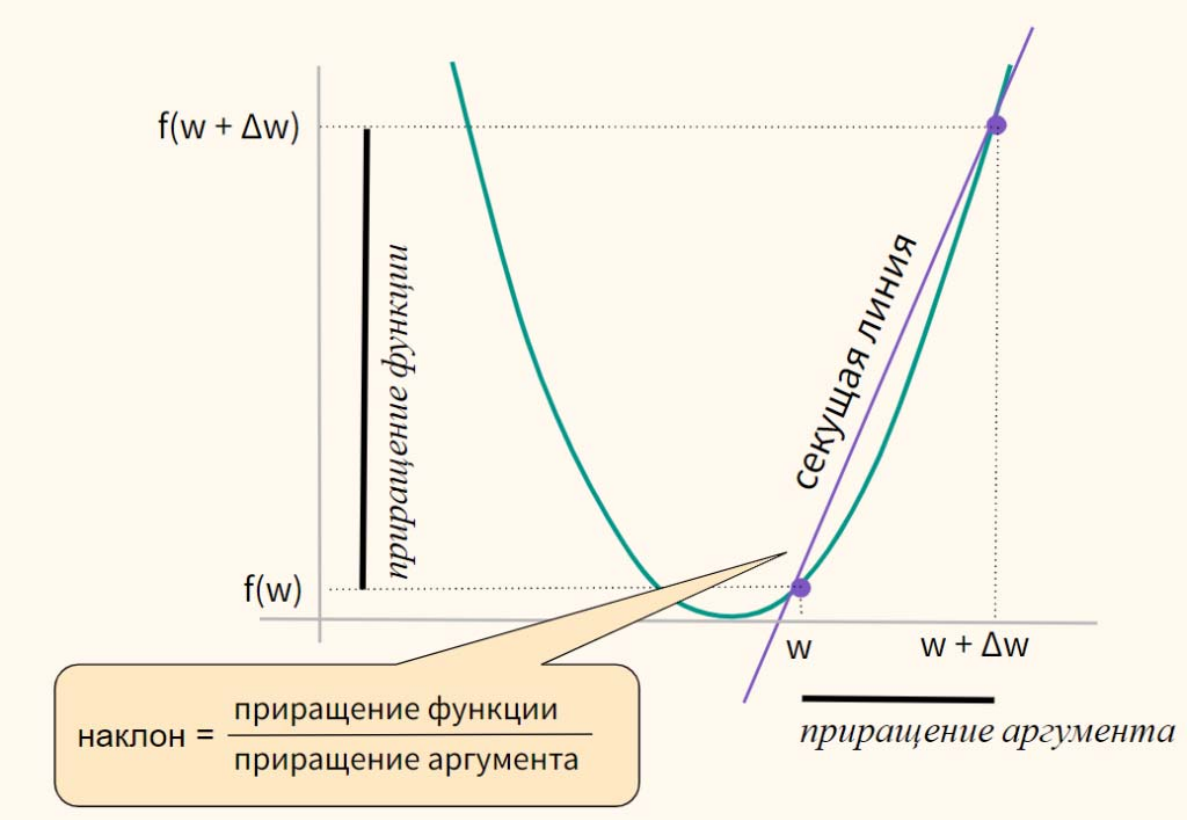
Число два характеризует наклон касательной к параболе в точке $w = 3$.

Нахождение производной

Остается выяснить, как мы пришли от функции к ее производной?

$$f(w) = (2 - w)^2 \to f'(w) = 2w - 4$$

Давайте проведем линию (она называется секущей, secant line) через нашу параболу в точке $(w; f(w))$ и в точке $((w + \Delta w); f(w + \Delta w))$. Здесь Δ — это заглавная греческая буква «дельта», которой часто обозначают разницу или изменение чего-либо.



> Наклон секущей линии будет равен **отношению приращения функции к приращению аргумента** (difference quotient).

Другими словами, мы посмотрим насколько изменится расстояние по вертикали $f(w + \Delta w) - f(w)$ при заданном изменении по горизонтали $(w + \Delta w) - w$ или просто Δw.

$$\frac{f(w + \Delta w) - f(w)}{\Delta w}$$

Графическое решение

Для того чтобы графически перейти от секущей к касательной, мы можем представить, что бесконечно уменьшаем приращение аргумента Δw. Ещё говорят,

что Δw стремится к нулю, $\Delta w \to 0$.

Так вот по мере того, как Δw стремится к нулю, секущая линия будет стремиться к касательной в точке w. Именно это нам и нужно.

Алгебраическое решение

Теперь давайте найдем производную параболы алгебраически. Для этого подставим в формулу выше нашу параболу, раскроем скобки и упростим выражение.

$$f'(w) = \frac{(2-(w+\Delta w))^2 - (2-w)^2}{\Delta w} =$$

$$\frac{(w^2 + 2w\Delta w + \Delta w^2 - 4w - 4\Delta w + 4) - (w^2 - 4w + 4)}{\Delta w} =$$

$$\frac{\cancel{w^2} + 2w\Delta w + \Delta w^2 - \cancel{4w} - 4\Delta w + \cancel{4} - \cancel{w^2} + \cancel{4w} - \cancel{4}}{\Delta w} =$$

$$\frac{\Delta w^2 + 2w\Delta w - 4\Delta w}{\Delta w} =$$

$$\frac{\cancel{\Delta w}\,(\Delta w + 2w - 4)}{\cancel{\Delta w}} =$$

$$2w - 4 + \Delta w$$

Вспомните, мы допустили превращение секущей в касательную при стремлении Δw к нулю. Так как это бесконечно малая величина, мы можем ей пренебречь, и у нас остаётся уравнение производной.

$$f'(w) = 2w - 4 + \cancel{\Delta w} = 2w - 4$$

Правила нахождения производной

Если у функции есть производная (а так бывает не всегда), то с помощью отношения приращений ее можно найти. Однако, по мере усложнения функции, такой способ нахождения производной становится очень долгим.

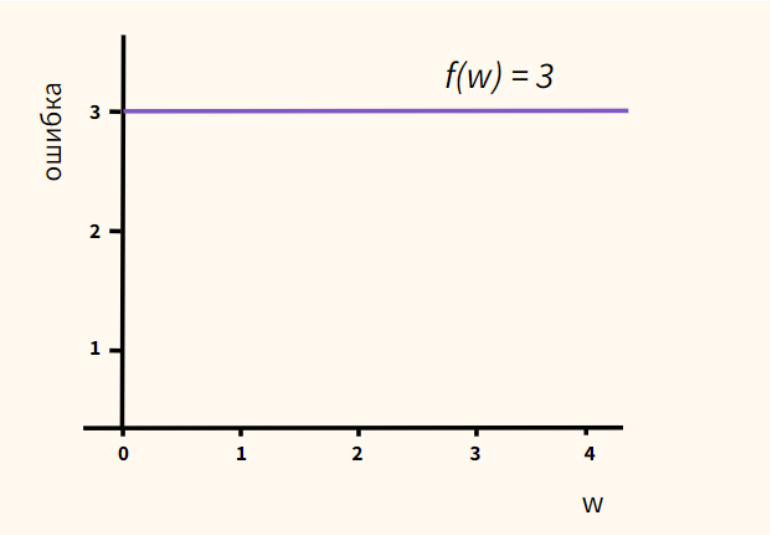
К счастью, из формулы отношения приращений можно вывести несколько несложных правил, которые упрощают и ускоряют процесс **дифференцирования функции** (т.е. нахождения ее производной).

Производная константы

График константы (C) — это всегда горизонтальная прямая, а значит наклон касательной в любой точке и производная равны нулю.

$$f(w) = C \to f'(w) = 0$$

Проиллюстрируем на графике.



Производная степенной функции

К степенной функции относится уже встречавшаяся нам квадратичная функция w^2. Ее можно дифференцировать по следующей формуле.

$$f(w) = w^n \to f'(w) = nw^{n-1}$$

Подставим число 2 вместо n.

$$f(w) = w^2 \to f'(w) = 2w^{2-1} = 2w$$

Умножение на число

Если функция умножена на число, то при дифференцировании, мы можем умножить производную функции на это число.

$$f(w) = Cg(w) \to f'(w) = Cg'(w)$$

Приведем пример.

$$f(w) = 2w^3 \to f'(w) = 2 \cdot 3w^{3-1} = 6w^2$$

Производная суммы и разности

Производная суммы двух функций равна сумме производных каждой из этих функций. То же самое с разностью.

$$(f(w) \pm g(w))' = f'(w) \pm g'(w)$$

Приведем несложный пример.

$$f(w) = w^4 + 3w \to$$

$$f'(w) = (w^4 + 3w)' = (w^4)' + (3w)' =$$

$$4w^{4-1} + 3w^{1-1} = 4w^3 + 3w^0 = 4w^3 + 3$$

Производная разности находится аналогично.

Производная произведения и частного

Производные произведения и частного требуют несколько более сложных вычислений, но в целом находятся аналогично производным суммы и разности двух функций.

$$(f(w) \cdot g(w))' = f'(w) \cdot g(w) + f(w) \cdot g'(w)$$

$$\left(\frac{f(w)}{g(w)}\right)' = \frac{f'(w) \cdot g(w) - f(w) \cdot g'(w)}{(g(w))^2}$$

Приведем примеры для каждой из формул. Начнем с произведения константы на степенную функцию.

$$f(w) = 5w^3$$

$$f'(w) = 0 \cdot w^3 + 5 \cdot 3w^2 = 15w^2$$

Замечу, что в данном случае мы пришли к тому же результату, как если бы использовали формулу умножения функции на число.

$$f'(w) = 5 \cdot 3w^{3-1} = 15w^2$$

Теперь посмотрим на производную частного двух функций.

$$f(w) = \frac{3}{2x}$$

$$f'(w) = \frac{0 \cdot 2w - 3 \cdot 2}{(2w)^2} = \frac{0-6}{4w^2} = -\frac{3}{2w^2}$$

Производная композиции функций

Прежде чем мы перейдем к нахождению производной композиции функций (composite function) или производной сложной функции, давайте в принципе вспомним, что это такое. Пусть даны две функции.

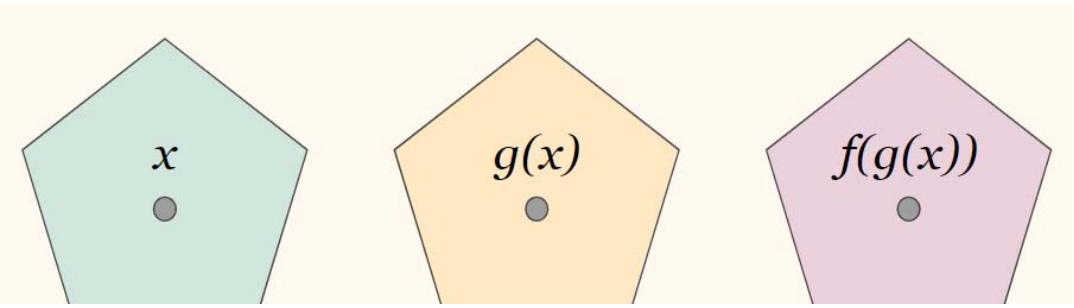
$$f(w) = w^3$$

$$g(w) = w - 1$$

Тогда композицией этих функций $f(g(x))$ будет

$$f(g(w)) = f(w-1) = (w-1)^3$$

На схеме это можно представить следующим образом:



Значение ***x*** вначале подается в функцию ***g***	Результат функции ***g*** затем подается в функцию ***f***

Если подставить значение в такую функцию (например, $x = 3$), то расчет будет выглядеть так.

$$f(g(3)) = f(3-1) = 2^3 = 8$$

Теперь про то, как дифференцировать такую функцию. Приведем правило.

$$(h(w))' = (f(g(w)))' = f'(g(w)) \cdot g'(w)$$

Давайте словами опишем каждое действие:

1. вначале дифференцируем внешнюю функцию $f(w)$;
2. затем вместо w подставляем внутреннюю функцию $g(w)$; получается сложная функция из производной внешней функции и внутренней функции $f'(g(w))$;
3. наконец мы умножаем сложную функцию на производную внутренней функции $g(w)$.

Эту формулу называют цепным правилом или **chain rule**.

Теперь найдем производную композиции функций, приведенных выше. Вначале для удобства вычислим производные каждой из функций.

$$f'(w) = 3w^2$$

$$g'(w) = 1w^{1-1} + 0 = w^0 = 1$$

Применим chain rule.

$$(f(g(w)))' = 3(w-1)^2 \cdot 1 = 3(w-1)^2$$

Онлайн-калькуляторы и библиотека SimPy

После того как вы поняли суть производной и принцип ее нахождения, в целях экономии времени удобно воспользоваться любым онлайн-калькулятором или Питоном.

Откроем блокнот к этому занятию

В Питоне можно использовать **библиотеку SymPy** (Symbolic Mathematics in Python). Библиотека SimPy — это система компьютерной алгебры (Computer Algebra System, CAS), которая позволяет работать с математическими выражениями, как если бы мы производили вычисления на бумаге.

Приведем распространенный пример с функцией квадратного корня. Если использовать, например, модуль **math**, то результатом извлечения квадратного корня из числа 11 будет его приближенное значение.

```
from math import sqrt
sqrt(11)
```

```
3.3166247903554
```

Если же использовать **SymPy**, то результатом будет математическое выражение.

```
from sympy import sqrt
sqrt(11)
```

$$\sqrt{11}$$

Для нахождения производной воспользуемся **функцией diff()**. Импортируем ее.

```
from sympy import diff
```

Кроме того, нам надо попросить Питон считать w математическим символом, а не обычной переменной, которая хранит какое-либо значение.

```
# для этого мы импортируем соответствующую букву из модуля abc
from sympy.abc import w
```

Остается прописать функцию, которую мы хотим дифференцировать, и передать ее в **diff()** вместе с символом w.

```
f = (2 - w) ** 2
diff(f, w)
```

$$2w - 4$$

Оптимизация функции и нахождение минимума

Итак, мы научились находить производную от функции и вычислять ее значение в заданной точке. Однако, и мы об этом уже сказали, нас интересует только та точка, в которой наклон касательной будет равен нулю, а значение функции минимально.

Аналитическое решение

Если мы приравняем уравнение производной к нулю и решим относительно w, то найдем минимум параболы.

$$2w - 4 = 0$$

$$2w = 4$$

$$w = 2$$

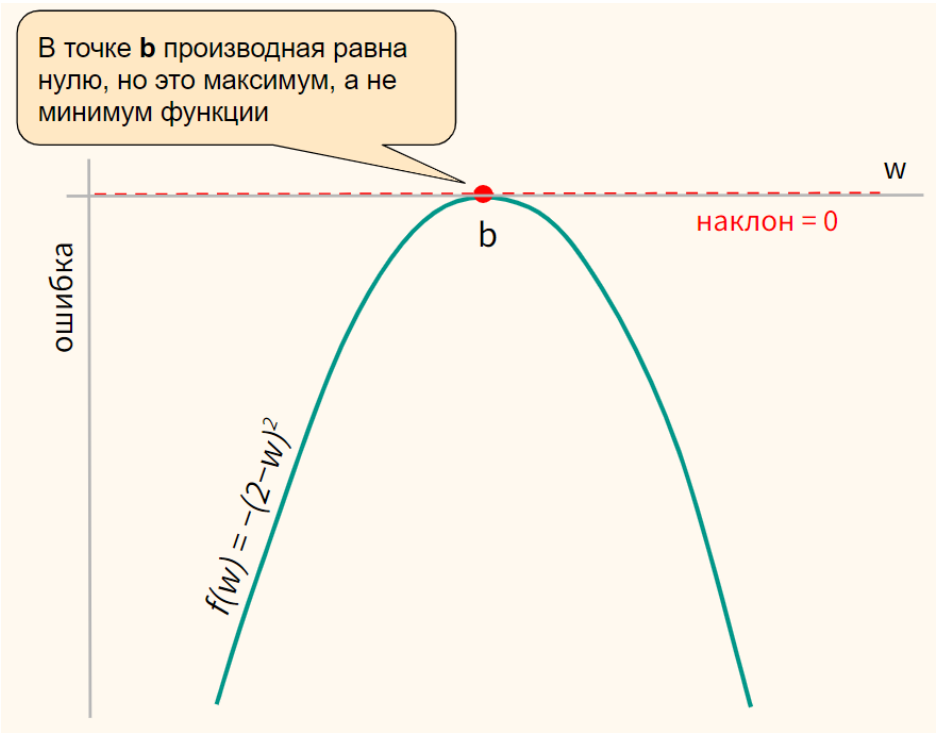
Как мы уже видели на первом графике, в точке $w = 2$ ошибка действительно минимальна.

К сожалению, такая простая оптимизация далеко не всегда возможна хотя бы потому, что наклон параболы (производная) равны нулю не только в точке минимума функции, но и в точке ее максимума.

Возьмем вот такую функцию:

$$f(w) = -(2 - w)^2$$

Ветви этой параболы направлены вниз и в точке, где наклон касательной равен нулю, находится ее максимум, а не минимум.

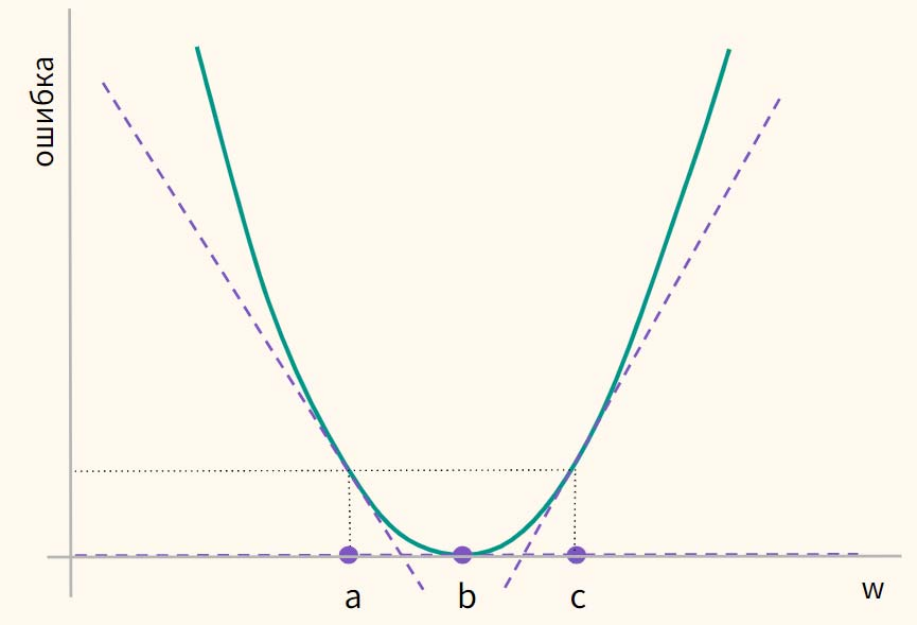


Похоже, что нам придется попросить компьютер подобрать для нас оптимальный вес w, при котором ошибка будет минимальна. Такой метод называется численным или вычислительным.

Численный метод оптимизации

Впрочем, с подбором весов с помощью компьютера есть одна сложность. Начальный вес, то есть наклон прямой линейной регрессии, выбирается случайным образом.

Например, это может быть точка a или точка c на графике ниже, где наклон касательной к функции потерь очевидно не равен нулю.



Как нам (или вернее комьютеру) узнать, в какую сторону сдвигаться по оси w, чтобы наклон касательной в результате стал равен нулю, а мы оказались в самой нижней точке? Простой перебор вариантов может оказаться нереализуемым на большом объеме данных и сложной функции потерь.

Здесь будет уместно привести *вторую* важную особенность производной функции.

> Помимо наклона, производная указывает **направление** скорейшего подъема по исходной функции.

Этот факт станет более очевиден, когда мы будем изучать функции с несколькими переменными на следующем занятии. Пока же, если мы согласимся с этим утверждением, то для того чтобы спуститься по функции вниз, нам нужно двигаться в направлении обратном направлению производной.

Пример численной оптимизации

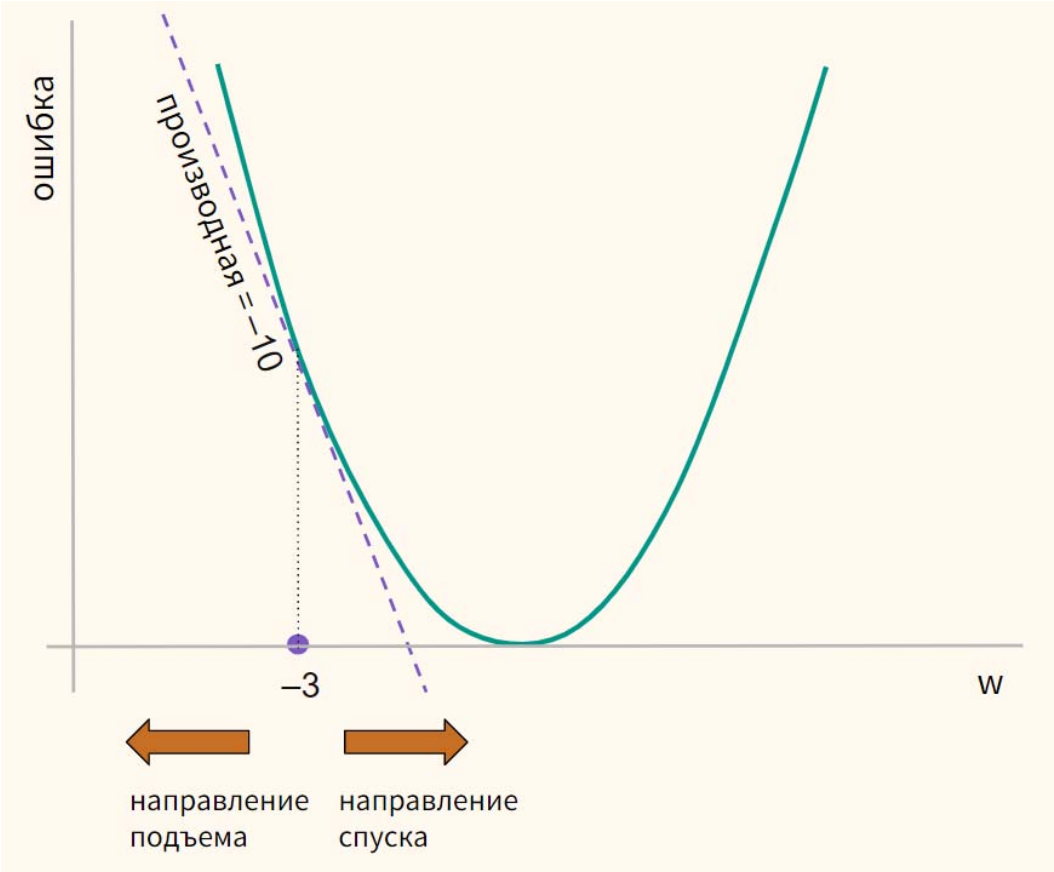
Вернемся к нашей функции потерь. Еще раз напомню саму функцию и ее производную.

$$f(w) = (2 - w)^2$$

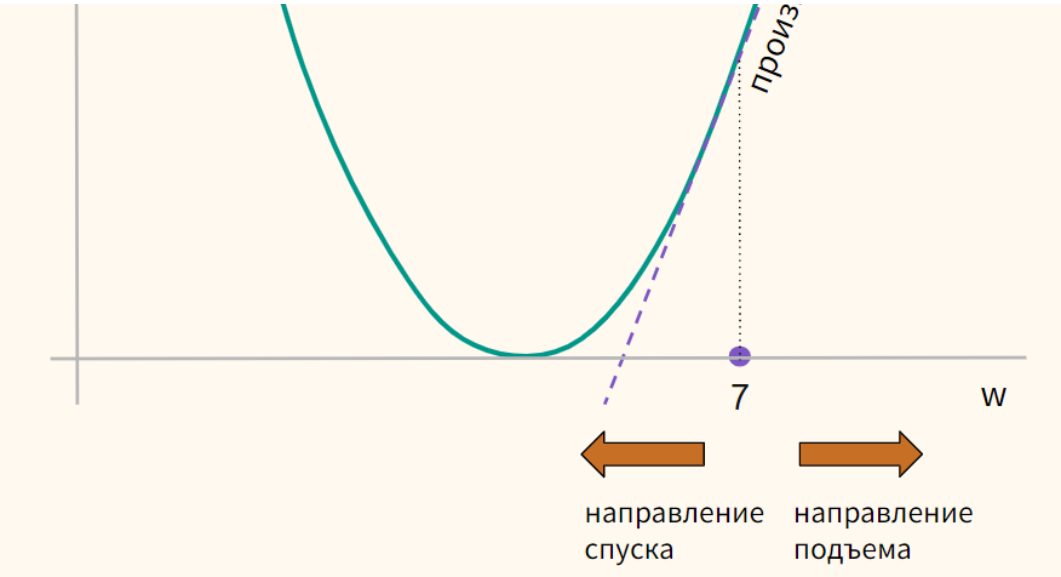
$$f'(w) = 2w - 4$$

Предположим, что мы выбрали начальный вес в точке $w = -3$. В этой точке производная равна

$$f'(-3) = 2 \times (-3) - 4 = -10$$



Для скорейшего подъема по функции вверх нам нужно двигаться «влево» на расстояние -10. Если же мы хотим (а мы как раз хотим) спускаться, то нам нужно двигаться «вправо», то есть на расстояние $-(-10)$ или $+10$. Сдвинувшись на это расстояние, мы окажемся в точке $w = 7$.



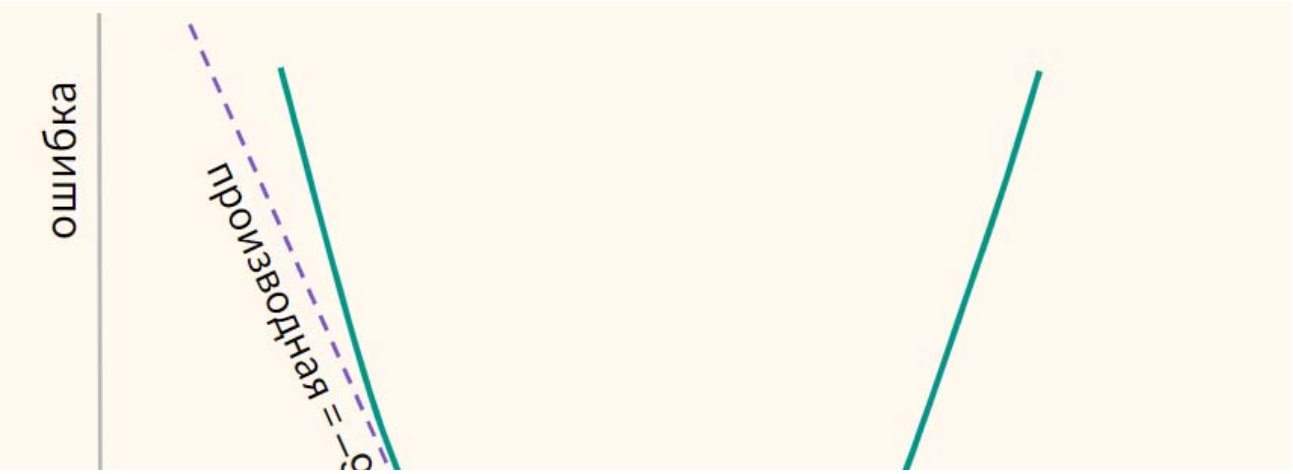
Все хорошо, только мы «перелетели» наш минимум. В результате теперь нам нужно двигаться в обратном направлении и, к сожалению, на такое же расстояние:

$$f'(7) = 2 \times 7 - 4 = 10$$

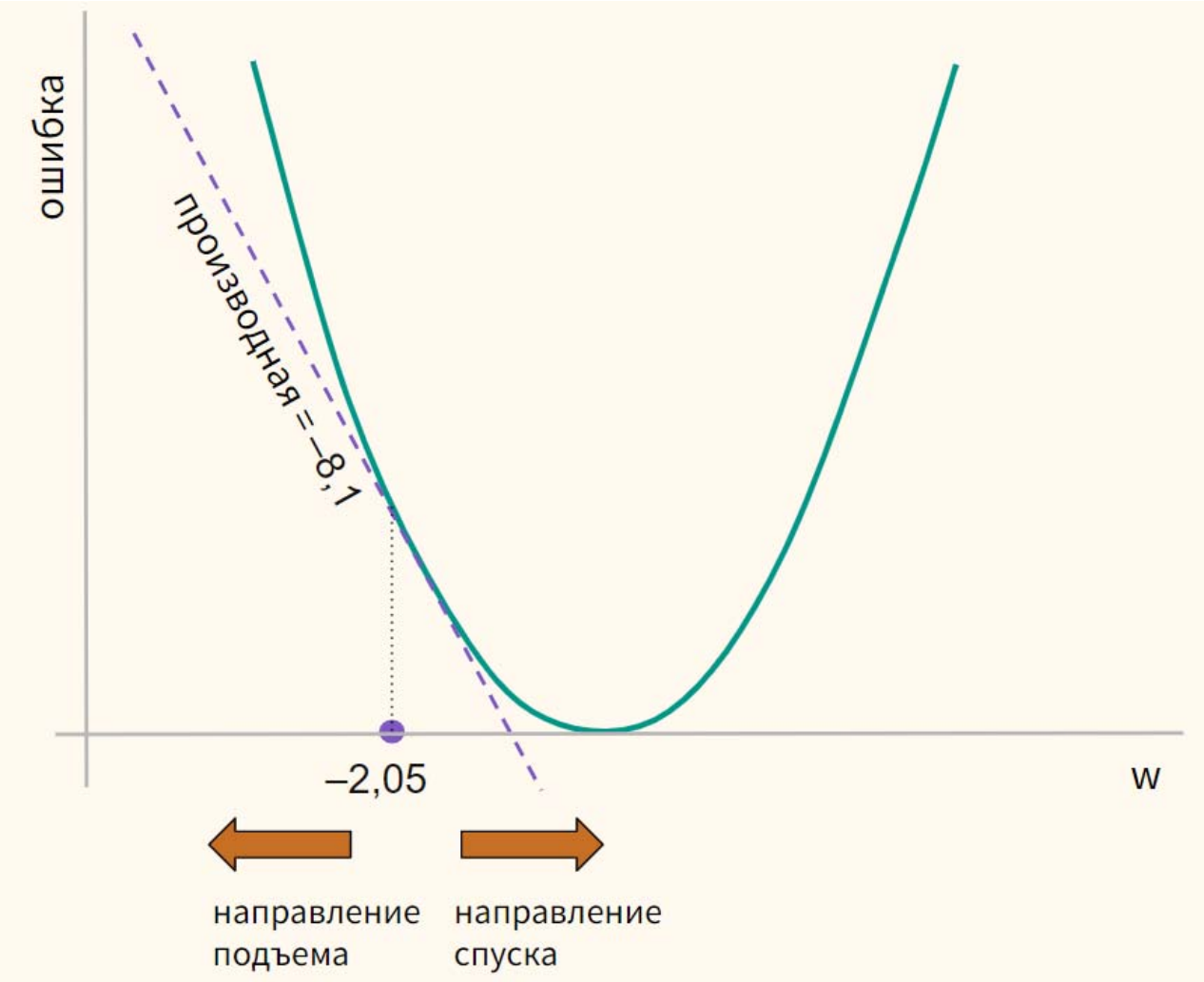
Давайте повторим это же действие, но предварительно умножим $-(-10)$ (обратное значение производной в точке $w = -3$) на коэффициент (назовем его «альфа», α), равный, например, $0{,}05$. Первый шаг «вправо» в этом случае будет равен не 10, а $10 \times 0{,}05 = 0{,}5$.

В результате мы окажемся в точке $w = -3 + 0{,}5 = -2{,}5$. Производная в этой точке равна

$$f'(-2.5) = 2 \times (-2.5) - 4 = -9$$



Тогда следующий шаг составит $9 \times 0{,}05 = 0{,}45$. Сместившись на это расстояние, мы окажемся в точке $w = -2{,}5 + 0{,}45 = -2{,}05$.



Так мы будем двигаться до тех пор, пока не окажемся «на дне» параболы в точке $w = 2$.

> Коэффициент α, который мы использовали для плавного спуска вниз по функции, принято называть **скоростью обучения** (learning rate). И действительно, в зависимости от значения α спуск будет происходить быстрее либо медленнее.

Дополнительно замечу, что если бы мы изначально оказались в точке, например,

$w = 3$, то двигались бы в обратную сторону («влево»).

Оптимизация функции на Питоне

Давайте реализуем этот несложный алгоритм на Питоне.

```
import numpy as np
import matplotlib.pyplot as plt
```

Вначале объявим функцию потерь и ее производную.

```
# функция потерь
def objective(x):
    return (x - 2 ) ** 2.0

# производная
def derivative(x):
    return 2.0 * x - 4
```

Теперь зададим исходные параметры алгоритма.

```
# изначальный вес линейной модели
w = -3

# количество итераций
iter = 100

# коэффициент скорости обучения
learning_rate = 0.05
```

Теперь создадим списки, в которые на каждой итерации будем записывать обновляемые веса (w_list), значение функции потерь при этих весах (l_list) и значение производной на каждой итерации (d_list).

```
# эти данные нам нужны только для оценки процесса оптимизации
w_list = []
l_list = []
d_list = []
```

Теперь напишем сам алгоритм.

```
# в цикле с заданным количеством итераций
for i in range(iter):

  # будем добавлять значение w,
  w_list.append(w)

  # уровень потерь
  l_list.append(objective(w))

  # и значение производной при текущем значении w
  d_list.append(derivative(w))
```

```
    # но главное, будем обновлять веса в направлении,
    # обратном направлению производной, умноженному на скорость обучения
    w = w - learning_rate * derivative(w)

# после завершения цикла, выведем итоговый вес w,
# значение функции потерь и значение производной в этой точке
w, objective(w), derivative(w)
```

```
(1.999867193005562, 1.76376977716448e-08, -0.00026561398887592347)
```

В процессе оптимизации мы вплотную приблизились к $w = 2$. Как и ожидалось, значение функции потерь, равно как и прозводная в этой точке, близки к нулю.

Дополнительно, давайте убедимся, что наши расчеты весов и производной, выполненные ранее «на бумаге», верны. Для этого выведем первые три значения из списков w_list и d_list.

```
w_list[:3], d_list[:3]
```

```
([-3, -2.5, -2.05], [-10.0, -9.0, -8.1])
```

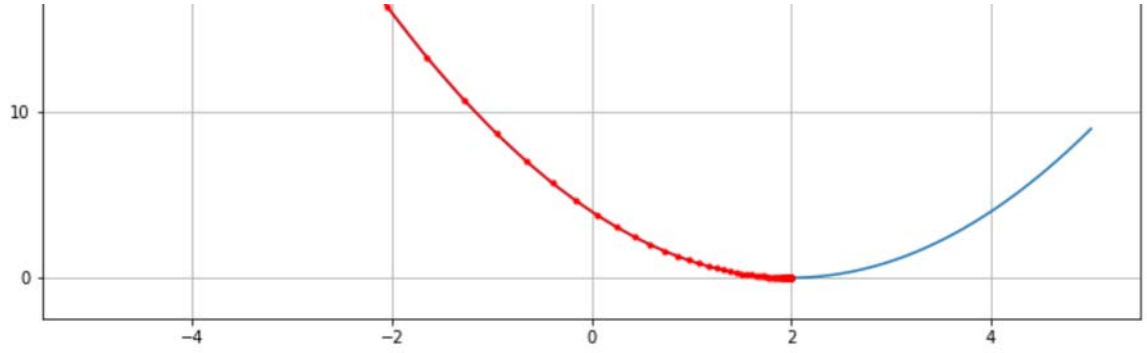
Все значения совпадают. Остается визуально оценить процесс «спуска».

```
# зададим размер графика
plt.figure(figsize = (12,10))

# построим функцию потерь (параболу)
x = np.linspace(-5, 5, 5000)
y = objective(x)
plt.plot(x, y)

# выведем шаги алгоритма оптимизации
plt.plot(w_list, l_list, '.-', color = 'red')

# добавим сетку и выведем результат
plt.grid()
plt.show()
```



Обратите внимание, вначале шаги были достаточно большими, но по мере приближения к точке минимума существенно сократились. Это связано с уменьшением «крутизны» или скорости изменения параболы и, как следствие, наклона касательной.

Это приводит нас к *третьей* важной особенности производной.

Производная показывает **скорость изменения функции** в определенной точке.

Прежде чем завершить, посмотрим как примерно менялся наклон прямой исходной модели по мере оптимизации коэффициента w.

```
# настроим размер графика
plt.figure(figsize = (12,10))

# создадим примерную последовательность наших весов от -3 до 2 и в цикле
for w in range(-3, 3):

  # создадим точки для координаты x,
```

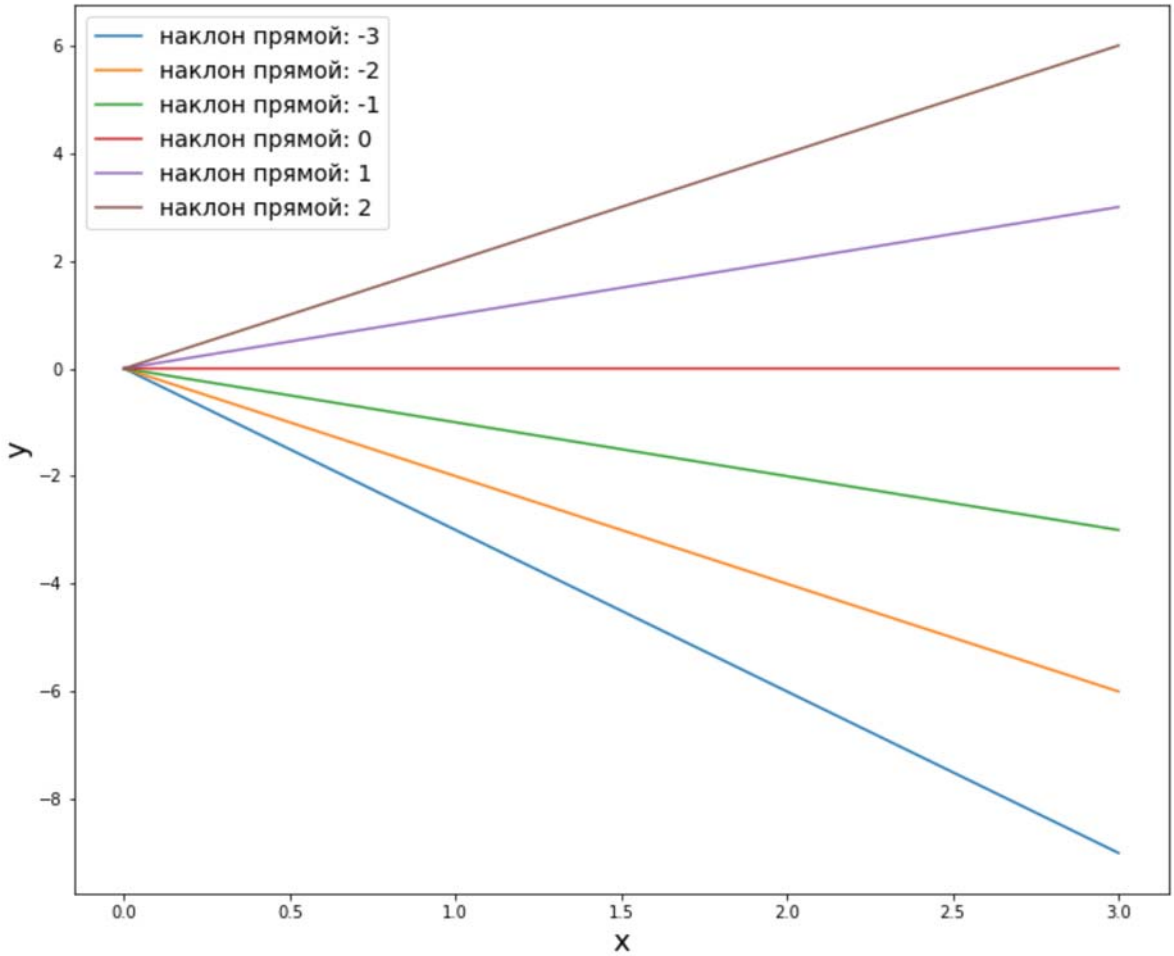
```
    x = np.linspace(0, 3, 5000)

    # умножим наклон w на x
    y = w * x

    # и построим прямые с соответствующим наклоном
    plt.plot(x, y, label = f'наклон прямой: {w}')

# укажем подписи к осям и положение легенды
plt.xlabel('x', fontsize = 18)
plt.ylabel('y', fontsize = 18)
plt.legend(loc = 'upper left', prop = {'size': 14})

# выведем результат
plt.show()
```



Когда у нас появятся реальные данные, процесс настройки модели станет еще более наглядным.

Подведем итог

В силу важности сегодняшнего занятия давайте еще раз по пунктам разберем весь пройденный материал:

- вначале у нас есть модель, которую мы хотим оптимизировать, у нее есть данные (признаки), веса этих признаков и результат (например, цена дома в задаче линейной регрессии);
- для оптимизации исходной модели (то есть поиска идеальных весов) мы берем вторую функцию, функцию потерь, которая на входе принимает разные варианты этих весов, а на выходе показывает насколько, при таких весах, мы отклонились от идеального описания исходных данных;
- идеальные веса находятся там, где функция потерь выдает минимальное значение («дно» параболы, например);
- зная значения производной от функции потерь в конкретной точке, мы можем спускаться в обратном ей направлении до тех пор, пока не дойдем до минимума;
- коэффициент скорости обучения, на который мы умножаем значение производной, позволяет регулировать длину шага при каждой итерации алгоритма.

При построении модели регрессии мы отказались от коэффициента сдвига. При этом простая линейная регрессия (simple linear regression), как известно, имеет вид

$$y = wx + b$$

Такую модель мы построим уже на следующем занятии. Однако прежде нам необходимо развить понятие производной и познакомиться с градиентом. Так мы перейдем к алгоритму, который называется методом градиентного спуска.